

# 笔记模式样式演示

xwysyy-typst · 2026 年 5 月

## 目录

1 第一章：标题与正文

1.1 二级标题样式

2 第二章：列表与强调

2.1 无序列表

2.2 有序列表

2.3 文本强调

3 第三章：代码与引用

3.1 行内代码

3.2 代码块

3.3 引用装饰

4 第四章：表格

5 第五章：链接与符号

5.1 链接

5.2 箭头符号

Bibliography

1

1

1

1

2

2

2

2

2

2

3

3

3

## 第一章：标题与正文

竞赛编程（Competitive Programming）是评估大语言模型代码能力的重要实验场。与日常软件开发中的代码补全不同，竞赛编程要求模型在严格的时间和空间约束下，独立完成从问题理解、算法设计到正确实现的全链路推理 [1], [2]。

### 二级标题样式

Description2Code [3]（2016）从 CodeChef、Codeforces 收集了 7764 道题目。APPS [1]（NeurIPS 2021）将规模扩展到 10000 题。CodeContests [2]（Science 2022）从三个源头汇聚了 13610 题，并引入时间分割来避免数据泄露。

### 三级标题样式

LiveCodeBench [4] 首创“时间滚动”机制——从 LeetCode、AtCoder 和 Codeforces 持续收集新题目。截至 2025 年 5 月，该数据集已积累超过 1055 道题目。

### 四级标题样式

这是四级标题下的正文段落，用来展示更深层级的样式效果。

## 第二章：列表与强调

### 无序列表

- ❖ 闭源模型通常优于开源模型，但差距在持续缩小

- ❖ 模型在 C++ 上的表现通常优于 Python [5]
- ❖ 动态规划、深度优先搜索等需要深层递归推理的问题类型，始终是模型的短板
  - ◇ 二级列表项：这是嵌套列表
  - ◇ 另一个二级列表项

有序列表

1. 第一步：收集题目
2. 第二步：生成测试用例
3. 第三步：评估模型能力

文本强调

这是 **加粗文本** 的效果。这是 红色强调 和 黄色强调。

第三章：代码与引用

行内代码

使用 `pass@k` 指标评估代码生成质量，也可以用 `Refine@K` 模拟多轮提交。

代码块

```
def solve(n: int, edges: list) -> int:
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
    return bfs(graph, 0)
```

引用装饰

这是一个引用装饰段落，用于突出重要的结论或观点。测试用例质量直接决定评估的有效性。

第四章：表格

Table 1: 主要代码生成评测基准对比

| Benchmark     | 年份   | 规模    | 发表      |
|---------------|------|-------|---------|
| APPS          | 2021 | 10K   | NeurIPS |
| CodeContests  | 2022 | 13.6K | Science |
| LiveCodeBench | 2024 | 1055+ | ICLR'25 |
| USACO         | 2024 | 307   | COLM    |
| CodeElo       | 2025 | 387   | arXiv   |

第五章：链接与符号

---

## 链接

详情参见 [Codeforces 官网](#)。

## 箭头符号

数据流：输入  $\rightarrow$  预处理  $\rightarrow$  模型推理  $\rightarrow$  输出

逻辑关系： $A \Rightarrow B$ ,  $B \Leftrightarrow C$

映射： $f \mapsto g$

## Bibliography

---

- [1] D. Hendrycks *et al.*, “Measuring Coding Challenge Competence with APPS,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. doi: [10.48550/arXiv.2105.09938](https://arxiv.org/abs/2105.09938).
- [2] Y. Li *et al.*, “Competition-Level Code Generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022, doi: [10.1126/science.abq1158](https://doi.org/10.1126/science.abq1158).
- [3] E. Caballero, OpenAI, and I. Sutskever, “Description2Code Dataset.” [Online]. Available: <https://zenodo.org/record/5665051>
- [4] N. Jain *et al.*, “LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=chfJJYC3iL>
- [5] S. Quan *et al.*, “CodeElo: Benchmarking Competition-Level Code Generation of LLMs with Human-Comparable Elo Ratings.” [Online]. Available: <https://arxiv.org/abs/2501.01257>